EXERCICE DE MAISON · JEREMIAH GEEKS · SEMAINE DU 14 NOVEMBRE

🥤 LE DISTRIBUTEUR

Quatre produits, un porte-monnaie, et **zéro bloc recopié**. La machine repropose son menu toute seule, encore et encore, tant qu'il te reste de l'argent. **Ton programme va être plus court que celui de la semaine dernière** — et il fera dix fois plus de choses.

`Thonny → Fichier → Nouveau → Enregistrer sous : distributeur_tonprenom.py`

🎯 **La règle :** la machine contient **quatre produits**, **un seul exemplaire** de chacun. Tu dis combien tu as d'argent. **Tant qu'il te reste au moins 100 F** et **qu'il reste quelque chose à acheter**, la machine réaffiche son menu. À la fin, elle imprime ton ticket.

✅ CE QUE TON PROGRAMME DOIT FAIRE

```
========================================
      LE DISTRIBUTEUR DE LA RECRE
========================================

Combien as-tu d'argent ? 400

----------------------------------------
 1. Chips - 100 F
 2. Jus - 150 F
 3. Chocolat - 200 F
 4. Gateau - 250 F
----------------------------------------
 Tu as 400 F.
 Ton choix (1 a 4) ? 3

   CLONK !  Chocolat tombe dans le bac.

  ( ... le menu revient tout seul : la
    case 3 affiche EPUISE, il te reste
    200 F ... )

 Ton choix (1 a 4) ? 4

   BZZZT !  Impossible : mauvais numero,
            produit epuise, ou pas assez
            d'argent.

  ( ... elle redemande. Tu prends le Jus,
    et la boucle s'arrete toute seule ... )

========================================
              TON TICKET
========================================
 Produits pris : 2 / 4
 Monnaie       : 50 F

 Plus assez d'argent. La recre est finie !
========================================
```

🧰 LA BOÎTE À OUTILS

La boucle :
`while argent >= 100 and achats < 4:`
les **deux points** à la fin, et **tout ce qui se répète est décalé** dessous.

Deux listes qui vont ensemble :
`produits = ["Chips", "Jus", ...]`
`prix = [100, 150, 200, 250]`
Le produit n° 3, c'est `produits[2]`, et il coûte `prix[2]`. **Même indice.**

Donc : `prix[choix - 1]`

`!=` veut dire **est différent de**.

⚠️ **Si tu oublies** `achats = achats + 1`, la boucle **ne s'arrête jamais**. Pour arrêter un programme fou : **Ctrl + C** dans la fenêtre du bas.

1 LA MACHINE

- Affiche le cadre de **40** signes `=` et le titre.
- Crée **deux listes**, dans le même ordre :

```
produits = ["Chips", "Jus", "Chocolat", "Gateau"]
prix     = [100, 150, 200, 250]
```

Les prix sont des **nombres** : pas de guillemets autour.

2 LE CLIENT

- Demande `Combien as-tu d'argent ?`, range dans `argent`, et **transforme en nombre**.
- Crée le compteur `achats` à **0**.

👉 **La suite est sur la page 2** : le cœur du programme, la fin, et les bonus.

3 LA BOUCLE — et le menu dedans

```
while argent >= 100 and achats < 4:

    ... à partir d'ici, TOUT est décalé de 4 espaces ...
    print(" 1. " + produits[0] + " - " + str(prix[0]) + " F")
    ... les trois autres lignes du menu ...
    print(" Tu as " + str(argent) + " F.")
    choix = input(" Ton choix (1 a 4) ? ")
    choix = int(choix)
```

Lis la condition : « tant qu'il me reste de quoi payer le **moins cher** (100) **et** qu'il reste un produit à prendre ». Les deux doivent être vraies pour continuer.

4 LA VENTE — toujours dans la boucle

Quatre conditions pour une vente. Lis-les une par une :

```
    if choix >= 1 and choix <= 4 and produits[choix - 1] != "EPUISE" and prix[choix - 1] <= argent:
        print("   CLONK !  " + produits[choix - 1] + " tombe dans le bac.")
        argent = argent - prix[choix - 1]    ← on paie
        produits[choix - 1] = "EPUISE"       ← le produit part
        achats = achats + 1                  ← le compteur AVANCE
```

Ces trois lignes vont ensemble. Si tu enlèves la dernière, la boucle tourne pour toujours : c'est **la** panne de la semaine.

5 LE REFUS

```
    else:
        print("   BZZZT !  Impossible : mauvais numero, produit epuise,")
        print("            ou pas assez d'argent.")
```

Remarque : en cas de refus, **rien ne change** — ni l'argent, ni le compteur. La boucle repart donc **exactement comme avant**, et redemande. C'est exactement ce qu'on veut (et exactement ce que fait notre jeu XO quand la case est prise).

6 LE TICKET — DEHORS de la boucle

Recolle ces lignes tout à gauche : elles ne doivent s'afficher **qu'une fois la boucle terminée**.

- `Produits pris : <achats> / 4`
- `Monnaie : <argent> F`
- **Si** `achats == 4` : `TU AS VIDE LA MACHINE. Respect.` **Sinon** : `Plus assez d'argent. La recre est finie !`

Le test qui ne trompe pas : si ton ticket s'affiche **après chaque achat**, c'est qu'il est resté décalé dans la boucle.

⭐ SI TU AS FINI — LES BONUS

- **Dis-lui ce qui manque.** Dans le refus : `print(" Il te manque " + str(prix[choix - 1] - argent) + " F.")`
- **Le bouton SORTIE.** Le client peut partir en tapant **0**. Il faut `choix = 1` **avant** la boucle, puis

  ```
  while argent >= 100 and achats < 4 and choix != 0:
  ```

  Pourquoi avant ? Parce que Python lit la condition **avant** le premier tour : si `choix` n'existe pas encore, c'est `NameError`.
- ⭐⭐ **Un cinquième produit.** Ajoute `"Biscuit"` et `50` à la fin des deux listes, une ligne de menu, et remplace les 4 par des 5. **Compte les lignes que tu as changées.** Sans boucle, il t'en aurait fallu dix fois plus : voilà ce que la boucle t'a donné.

Quand ça marche : fais **trois parties** — une où tu vides la machine (il faut 700 F), une où tu tombes en panne d'argent, et une où tu demandes un produit **déjà épuisé**. Les trois doivent se terminer proprement sur un ticket. **Ctrl + S**, et **envoie une photo de ton écran** à ton enseignant.
La panne de la semaine : la boucle ne s'arrête plus. Dans 9 cas sur 10, `achats = achats + 1` manque, ou il est **collé à gauche** au lieu d'être dans le `if`. **Ctrl + C** arrête le programme, et on recompte les espaces.